

How to back up a live SQLite database

John Saitall

The order system at Quillmont Coffee keeps every order in one SQLite file, `orders.db`, and writes to it all day. Copying that file while the application runs looks like a backup, but the copy can be incomplete or even corrupt. This guide shows two safe ways to back up a database that is in use, how to check the result, and how to restore it.

Before you begin

You need:

- The `sqlite3` command-line shell, version 3.27.0 or later, because `VACUUM INTO` first appeared in 3.27.0. Run `sqlite3 --version` to check.
- Read access to the database folder and write access to a backup folder.
- Free disk space at least the size of the database.

All commands run from the folder that holds `orders.db` and write to a `backups` subfolder. To practice on a throwaway database first, build one with the script in the [appendix](#).

Why a file copy isn't a backup

Most applications that read and write at the same time run SQLite in *WAL mode* (write-ahead logging). New transactions go to a second file, `orders.db-wal`, and move into `orders.db` only later, at a *checkpoint*. Until then, the main file doesn't contain them.

Here is what a plain copy produced while the application was taking orders:

File	Orders
Live database (<code>orders.db</code> plus its WAL file)	5,037
A copy of <code>orders.db</code> alone	5,000

The copy missed every order placed since the last checkpoint, and nothing warned about it. In the older *rollback journal* mode, the risk is worse: a copy taken in the middle of a write can capture half of a transaction, which leaves a corrupt file.

The copy didn't fail. It just quietly left out the newest orders.

Choose a method

Both methods read the database inside a single read transaction, so the backup is a consistent snapshot of one moment. In WAL mode, the application keeps writing while the backup runs.

	<code>.backup</code>	<code>VACUUM INTO</code>
What it is	A command of the <code>sqlite3</code> shell	An SQL statement, so it also runs from application code
Output file	A page-by-page copy	A rebuilt copy without free pages, often smaller
Journal mode of the copy	Same as the source (WAL here)	Rollback journal (delete)
If the target file exists	Overwrites it without warning	Refuses: output file already exists
Best for	Scheduled backups from a script	Backups started by the application, compact archive copies

Back up from the shell

1. Run `.backup` with a date in the file name, so today's backup doesn't overwrite yesterday's.

In PowerShell:

```
sqlite3 orders.db ".backup 'backups/orders-$(Get-Date -Format yyyy-MM-dd).db'"
```

In Bash:

```
sqlite3 orders.db ".backup 'backups/orders-$(date +%F).db'"
```

2. Verify the backup.

Warning: `.backup` replaces an existing file of the same name without asking. A fixed name such as `backups/orders.db` keeps only the latest backup, including a bad one.

Back up with SQL

1. Run the statement with a target file that doesn't exist yet:

```
sqlite3 orders.db "VACUUM INTO 'backups/orders-2026-10-09-compact.db';"
```

2. Verify the backup.

To back up from application code, run the same `VACUUM INTO` statement through the application's own database connection.

Verify the backup

A backup that nobody has opened is a hope, not a backup. Check every new file before you rely on it.

1. Check the file's internal structure:

```
sqlite3 backups/orders-2026-10-09.db "PRAGMA integrity_check;"
```

The only acceptable output is `ok`.

2. Compare the row counts of the backup and the live database:

```
sqlite3 backups/orders-2026-10-09.db "SELECT count(*) FROM orders;"
sqlite3 orders.db "SELECT count(*) FROM orders;"
```

The backup can have fewer rows than the live database, because orders keep arriving after the backup is taken. It shouldn't have *more*.

Restore a backup

Restoring replaces the whole database, so every order placed after the backup is lost. Save the current state first, even if you think it's broken.

1. Stop the application, so that nothing writes to `orders.db` during the restore.
2. Back up the current database:

```
sqlite3 orders.db ".backup 'backups/orders-before-restore.db'"
```

3. Restore the backup into the live database:

```
sqlite3 orders.db ".restore 'backups/orders-2026-10-09.db'"
```

4. Run `PRAGMA integrity_check;` on `orders.db` and compare its row count with the backup's, as in [Verify the backup](#).
5. Start the application.

Note: Use `.restore` instead of copying the backup file over `orders.db`. A copied file can end up next to an old `orders.db-wal`, and SQLite then pairs the restored database with a log file from a different one. SQLite's own documentation lists this as a way to corrupt a database.

Troubleshooting

Problem	Fix
Error: database is locked	Another connection holds a write lock; this happens in rollback journal mode. Make the shell wait for it: <code>sqlite3 orders.db ".timeout 10000" ".backup 'backups/orders-2026-10-09.db'"</code>
output file already exists	<code>VACUUM INTO</code> doesn't overwrite. Choose a new file name, or delete the old file if you no longer need it.
The backup has fewer rows than the live database	Expected: these are orders placed after the backup. To confirm, compare <code>SELECT max(placed) FROM orders;</code> in both files.
<code>PRAGMA integrity_check</code> returns anything but ok	Don't keep the file. Take a new backup and check it. If the live database fails the check too, restore the last backup that passed.
<code>sqlite3</code> is not recognized as a command	The shell isn't installed or isn't on your <code>PATH</code> . Install it from your package manager, and open a new terminal.

Appendix: a sample database

This script creates an orders.db in WAL mode with 5,000 sample orders. Save it as create-sample.sql, then run `sqlite3 orders.db < create-sample.sql` in Bash. PowerShell has no < redirection; there, run `Get-Content create-sample.sql | sqlite3 orders.db`.

```
PRAGMA journal_mode = WAL;
```

```
CREATE TABLE orders (  
  id      INTEGER PRIMARY KEY,  
  customer TEXT NOT NULL,  
  product TEXT NOT NULL,  
  qty     INTEGER NOT NULL,  
  placed  TEXT NOT NULL DEFAULT (datetime('now'))  
);  
  
WITH RECURSIVE n(i) AS (SELECT 1 UNION ALL SELECT i + 1 FROM n WHERE i < 5000)  
INSERT INTO orders (customer, product, qty)  
SELECT 'Customer ' || (i % 250),  
       CASE i % 3 WHEN 0 THEN 'Espresso blend 1 kg'  
                WHEN 1 THEN 'Filter blend 500 g'  
                ELSE 'Decaf 250 g' END,  
       1 + i % 4  
FROM n;
```

The script prints wal once, confirming the journal mode. `SELECT count(*) FROM orders;` then returns 5000.